

智慧食材辨識系統

使用說明手冊

YOLO11 食材偵測 × 規則式推薦 × 三層 LLM 容錯鏈
FastAPI 後端 + Streamlit 前端（含攝影機即時辨識）

完整操作手冊 + 技術架構

一、系統簡介

本系統可辨識照片或攝影機畫面中的食材，並依現有食材推薦合適的家常食譜，再由 AI 生成親切的料理建議。

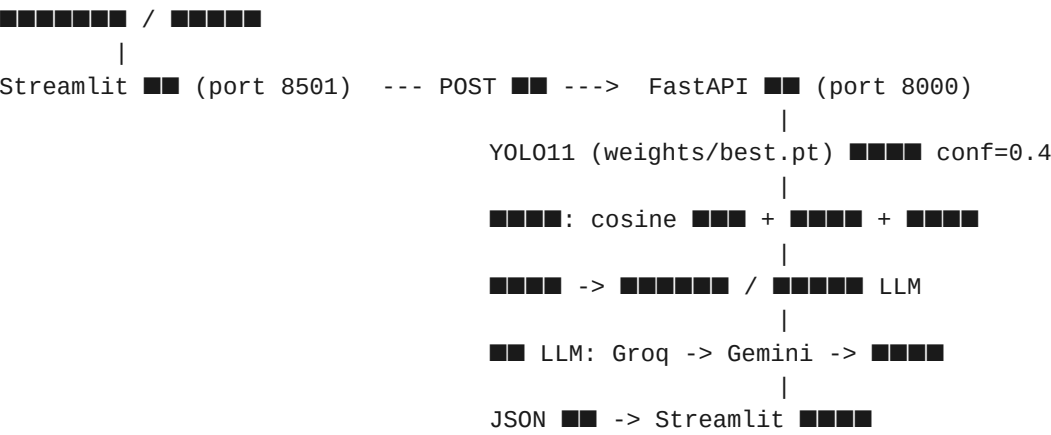
核心流程：使用者提供食材影像 → YOLO11 偵測食材 → 推薦引擎比對食譜 → LLM 生成推薦說明 → 顯示結果。

主要功能

- 上傳圖片辨識：上傳食材照片，取得辨識結果與推薦食譜。
- 攝影機即時辨識：開啟鏡頭即時顯示 YOLO Bounding Box，對準後拍照分析。
- 食譜推薦：依食材相符度排序，標示缺少的主食材。
- AI 料理建議：三層容錯鏈確保永遠有回應。

二、系統架構

系統分為前端（Streamlit）、後端（FastAPI）、模型與資料三大部分，資料流如下：



元件對照

元件	技術	說明
前端	Streamlit	兩分頁：上傳圖片 / 攝影機即時辨識
後端 API	FastAPI + Uvicorn	/detect-and-recommend、/health
偵測模型	YOLO11s (Ultralytics)	13 類食材，mAP@0.5 約 0.905
推薦引擎	scikit-learn cosine	食材向量相似度 + 加權評分
LLM 服務	Groq / Gemini / 模板	三層容錯，永不失敗
快取	記憶體 FIFO	相同食材組合不重複呼叫 LLM

三、環境需求與安裝

環境需求

- 作業系統：Windows（macOS / Linux 亦可，啟動指令略有不同）

- Python：3.11 以上
- GPU（建議）：NVIDIA GPU + CUDA（可選，CPU 亦可執行）
- 攝影機功能：需本機有可用 webcam

※ 本庫已附訓練好的 weights/best.pt，clone 後安裝套件即可執行，不需重新訓練或下載資料集。

安裝步驟（從 GitHub clone）

1. 取得專案：

```
git clone https://github.com/flykingjve/Food-detect-AI-recipe-recommendation.git
cd Food-detect-AI-recipe-recommendation
```

2. 建立並啟動虛擬環境（首次）：

```
python -m venv venv
venv\Scripts\activate
```

3. 安裝套件（預設 CPU 版 torch；GPU 另裝對應 CUDA 版）：

```
pip install -r requirements_backend.txt
:: GPU ■■■CUDA 12.4■■■
:: pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
```

4. 設定 API 金鑰（從範本複製後填入自己的金鑰）：

```
copy .env.example .env
:: ■■■ .env ■■■ GEMINI_API_KEY ■ GROQ_API_KEY
```

四、啟動系統

需開兩個終端機，先啟動後端，再啟動前端。

終端機 A — 後端

```
uvicorn app.main:app --reload --port 8000
```

成功會看到“YOLO11 已載入”與“食譜資料庫：N 道”。

- API 文件：<http://localhost:8000/docs>
- 健康檢查：<http://localhost:8000/health>

終端機 B — 前端

```
venv\Scripts\activate
streamlit run frontend.py --server.port 8501
```

瀏覽器開啟 <http://localhost:8501> 即可使用。

※ 手機同網路測試：加上 `--server.address 0.0.0.0`，手機連 <http://<電腦IP>:8501>。

五、功能操作

分頁一：上傳圖片

- 點選「上傳食材圖片」，選擇 jpg / jpeg / png 檔。
- 系統顯示辨識到的食材、AI 料理建議與推薦食譜（依相符度排序）。
- 每道食譜可展開查看主食材、所需時間、難度與缺少的食材。

分頁二：攝影機即時辨識

- （可選）設定「攝影機編號」，多顆鏡頭時切換，預設 0。
- 勾選「開啟即時辨識」：畫面即時顯示 YOLO Bounding Box 與當前偵測到的食材名稱。
- 對準食材後按「拍照並分析」：擷取當下畫面送後端，顯示推薦食譜與 AI 建議。
- 想再次即時辨識，重新勾選「開啟即時辨識」即可。

※ 即時畫面只顯示食材；完整推薦與 AI 建議在拍照後才產生。

六、食譜資料庫生成

推薦功能需要食譜資料庫 data/recipes.json。可用 Gemini 批次生成：

```
python scripts\generate_recipes.py
```

- 資料庫為空時系統仍可啟動與偵測，只是推薦清單為空。
- 若出現 429（配額用盡），腳本會存下已成功的部分，待配額重置後再執行續補。

※ 各模型免費額度、RPM、RPD 以官方 Dashboard 即時顯示為準。

七、技術架構詳解

7.1 YOLO11 食材偵測

使用 Ultralytics YOLO11s，輸入影像、conf=0.4，輸出框與類別。類別 id 對照（與 dataset.yaml 字母順序一致）：

id	食材	id	食材	id	食材
0	Beef 牛肉	5	Egg 蛋	10	Potato 馬鈴薯
1	Cabbage 高麗菜	6	Eggplant 茄子	11	Radish 蘿蔔
2	Carrot 紅蘿蔔	7	Leek 蒜苗/蔥	12	Tomato 番茄
3	Chicken 雞肉	8	Onion 洋蔥		
4	Cucumber 小黃瓜	9	Pork 豬肉		

7.2 推薦引擎

將食譜的食材轉成 0/1 向量，計算使用者食材與各食譜的 cosine 相似度，再加權：

- 主料加權：命中主食材每項 +0.15
- 缺料懲罰：每缺一項主食材 -0.10
- 接近獎勵：僅缺 1 項主食材 +0.05

最終分數排序取前 5 名，並標示缺少的主食材與相符度百分比。

7.3 三層 LLM 容錯鏈

依序嘗試，前一層失敗自動降到下一層，確保永遠有回應：

層級	服務 / 模型	角色
L1	Groq llama-3.1-8b-instant	主力，速度快

層級	服務 / 模型	角色
L2	Gemini gemini-3.1-flash-lite	備援
L3	靜態模板	永不失敗的保底文字

離線食譜生成另用 Gemini gemini-3.5-flash。

※ 模型配額（RPM / RPD / 免費額度）一律以官方 Dashboard 即時顯示為準。

7.4 快取機制

以「排序後的食材組合 + 食譜 id」為鍵的記憶體 FIFO 快取（上限 500），相同情境不重複呼叫 LLM，加快回應並節省配額。

八、常見問題排除

問題	可能原因與解法
check_env 顯示無 GPU / torch 為 CPU 版	安裝了 CPU 版 PyTorch。重裝 CUDA 版：pip install torch --index-url https://download.pytorch.org/whl/cu124
攝影機「無法開啟」	鏡頭未連接或被其他程式占用；關閉視訊軟體後重試，或調整「攝影機編號」。
前端顯示「無法連線後端」	FastAPI 未啟動。先在終端機 A 執行 uvicorn app.main:app --port 8000。
AI 建議一直是制式句 / 食譜生成 429	Gemini 金鑰無效或配額用盡。確認金鑰為 AIzaSy... 格式；配額以官方 Dashboard 為準，重置後再試。
即時畫面顏色不對（紅藍對調）	顯示需指定 channels=BGR（已於 frontend.py 處理）。
食材常漏偵測	將 app/main.py 的 conf=0.4 調低為 0.3。

九、專案結構（重點）

```

ai food detect project\
  app\                ■■■main.py / recommendation.py / llm_service.py / cache.py
  scripts\            generate_recipes.py■■■■■■■
  data\recipes.json   ■■■■■■
  weights\best.pt     YOLO11 ■■■■■■
  frontend.py         Streamlit ■■■■■■ + ■■■■■■
  setup\venv\         Python ■■■■■
  requirements_backend.txt / .env / README_backend.md

```

資料集：NutriLens Food Ingredients Detection（CC BY 4.0）